第4回　Markdownで原稿を組み立て、Word・PDF・slideへ変換する

記法、数式、Pandoc、Marp

2026年10月13日（火）3限　金融論

文書とslideを作る

原稿・図・式を含む短い文書を作り、HTMLまたはWord、MarpのPDFへ変換します。変換の指定はCodexに任せ、出力で文字・数式・図のcaption・長い見出しを確認します。

図の代替textは読み上げや変換先のcaptionにも使われます。CSVの末尾を練習コピーで変えると、図だけでなく本文の値・期間・結論を再点検する箇所が分かります。

この回で扱うこと

課題2以降の原稿は、Markdownで書くことが多いです。この回で扱うのは、その書き方と、そこからWord・PDF・slideを作る方法です。

- 文章を扱うfile形式の整理
- Markdownの記法と、日本語で書くときに引っかかるところ
- 数式の書き方
- Pandocで `.docx` とHTMLに変換する
- 日本語PDFの作り方（失敗しやすい箇所が決まっています）
- Marpでslide用の原稿からPDFを作る

変換commandの細かい指定はCodexに組ませます。

配布物は2つあります。`workbook/workbook.md` は、上から順に手を動かしていくワークブックです。授業中はこちらを開いてください。`sample/` は記法を一通り使った完成原稿で、`report.md` と `slides.md`、それぞれの変換後のfileが入っています。

文章のfile形式

文章を扱うfile形式は、中身の作りで3つに分かれます。

そのまま読めるtext

editorで開けば中身が見えます。行単位で差分が取れるので、AIが何を書き換えたかを追えます。

拡張子	何か
`.txt`	ただの文字。構造を示す方法がありません
`.md`	Markdown。`#` や `-` などの記号で構造を示します
`.tex`	LaTeX。組版の指定まで書きます。論文誌の原稿
`.html`	webページ。タグで構造を示します
`.ipynb`	Jupyter notebook。文章・code・実行結果が1つのfileに入ります。
`.rtf`	書式付きtext。Wordより前からある互換用の形式です

中身を固めたもの

1つのfileに見えますが、実体は複数のfileをZIPで固めたものです。差分は取れません。

拡張子	何か
`.docx`	Word
`.pptx`	PowerPoint
`.odt`、`.odp`	LibreOfficeなど。`.docx`・`.pptx` に対応する公開規格
`.pages`、`.key`	Apple の Pages と Keynote
`.epub`	電子書籍。中身はHTMLの集まり

Excelの `.xlsx` も同じ作りです。Google DocsやGoogle Slidesはfileではなくservice上の文書で、`.docx` や `.pptx` に書き出して受け渡しします。

配布用

`.pdf` は、font・改行位置・図の配置まで固定した形式です。どの環境でも同じに見える代わりに、編集は想定されていません。提出と印刷のための最終形と考えます。

この授業で使うもの

原稿を `.md` で書き、提出する形に変換します。

用途	使うもの
書く	`.md`
提出する	`.docx`、`.pdf`
発表する	Marpのslide（`.pdf`）
卒論や論文誌	`.tex`（補論で触れます）

なぜMarkdownで書くか

原稿はMarkdownで書き、提出する形式には最後に変換する、という順番の話です。提出先や共著者がWordを求める場面は多いので、変換は後ろに置きます。

`.docx` の正体

さきほど「中身を固めたもの」と書いた `.docx` を、実際に開いてみます。拡張子を `.zip` に変えると中が見えます。

```
$ unzip -l report.docx
  1933  [Content_Types].xml
   722  _rels/.rels
  4681  word/document.xml
  1203  word/_rels/document.xml.rels
 27124  word/styles.xml
   843  word/footnotes.xml
```

本文は `word/document.xml`、書式は `word/styles.xml`、脚注は `word/footnotes.xml` と、役割ごとに分かれています。これらをZIPで固めたものが `.docx` です。通常のtext diffでは圧縮前の本文を直接読めません。Wordには文書比較機能があります。Markdownは通常のtext diffで変更行を読みやすい形式です。

Markdownはただのtextです。AIが直接読み書きでき、変更は行の差分として出ます。AIが何を書き換えたかを、出力ではなく差分で見られます。

一つの原稿から複数の形式

`report.md` から `.docx`、HTML、PDFを作れます。slideは別の `slides.md` から作ります。どちらもMarkdownを元の原稿にすると、変更を差分で確かめ、出力を作り直せます。

図をscriptで作って、Markdownからrelative pathで参照しておくと、dataを更新して図を作り直すだけで済みます。画像pathはそのまま使えます。caption・期間・本文の数値・結論は更新後に照合します。Wordに図を貼り込んだ場合は、更新のたびに貼り直しになります。

見た目ではなく構造を書く

Wordの直接書式と見出しstyleは別です。見出しstyleを使えばWordでも構造を表せます。Markdownの `##` は「見出しレベル2」という構造です。`.docx` ではWordの見出しstyle、HTMLでは `<h2>` になります。体裁は変換のときに決まります。

記法

一覧

書くもの	記法
見出し	`# 大見出し`、`## 中見出し`、`### 小見出し`
強調	`**太字**`、`*斜体*`
箇条書き	行頭に `-`。番号付きは `1.`
link	`[表示する文字](https://example.com)`
画像	`![代替text](figures/fig01.png)`
引用	行頭に `>`
文中のcode	`` `pandoc` `` のようにbacktickで挟む
code block	3つのbacktickで囲む。先頭に `python` などの言語名
脚注	本文に `[^1]`、別の行に `[^1]: 脚注の中身`
区切り線	`---` を単独の行に

表は縦棒で区切り、2行目に `|---|---|` を置きます。縦棒の位置が揃っていなくても動きます。

```
| 系列 | 出所 |
|---|---|
| 実質GDP | 内閣府 |
| CPI | 総務省統計局 |
```

段落と改行

空行が段落の区切りです。日本語の文章は1段落を1行で書くと、変換後に余分な空白が入りません。

道具によって使える記法が違う

Markdownには方言があります。脚注はPandocでは使えますが、Marpのslideでは扱いが違います。GitHubの画面で見えるものとPandocの出力が一致しないこともあります。どの道具に通すかで、使える記法が決まります。

VS Codeのプレビュー（⇧⌘V）を元のtextと並べて開くと、記号と見た目の対応が読めます。
`sample/report.md` はこの表の記法を全部使っていて、末尾にどこで使ったかの対応表を付けてあります。

数式

書き方

数式はLaTeX記法で書きます。文中は `$` で挟み、独立した行は `$$` で囲みます。

```
文中：収益率は $r_t = (P_t - P_{t-1})/P_{t-1}$ で定義される。

独立行：
$$
i_t = r^* + \pi_t + a(\pi_t - \pi^*) + b \cdot \text{gap}_t
$$
```

金融論でよく出てくるのは、複利 $(1+r)^n$、現在価値 $\sum_t C_t/(1+r)^t$、収益率 $r_t = (P_t - P_{t-1})/P_{t-1}$ あたりです。添字は `_`、べき乗は `^`、2文字以上は `{}` でくくります。

変換後は数式の表示を確かめます。`.docx` ではWord上で数式を編集できます。

AIが出した式をそのまま貼ると、記号の定義が本文になかったり、添字が途中で変わっていたりします。式を書いたら、記号が何を指すかを本文の側に書いておくと、後から読み直せます。

原稿の組み立て

front matter

fileの先頭に `---` で囲んだ部分を置くと、Pandocが題名・著者・日付として読みます。

```
---
title: "日米自動車産業と為替"
author: "久保田荘"
date: "2026年10月15日"
---
```

見出しの階層

`#` が章、`##` が節です。`--toc` を付けて変換すると、この階層から目次が自動で作られます。章を飛ばして `###` から始めると、目次の階層が崩れます。

図と表

図には番号とcaptionを付けて、本文から参照します。pathは原稿fileからの相対で書きます。

```
![図1　日米の政策金利](figures/fig01_policy_rates.png)
```

`figures/` を原稿と同じfolderに置いておくと、folderごと移動しても壊れません。絶対pathで書くと、他の人のPCでは開けません。

骨格

課題の原稿は、題名、要約、背景、data、分析、結論、referencesの順に組みます。中身を書く前に空の見出しだけ並べておくと、どこが埋まっていないかが目次に出ます。

構成案はAIに出させると早いです。ただし、その章立てにした理由は自分の言葉で書けるようにしておくと、後で構成を変えるときに判断できます。

WordとHTMLへの変換

Pandoc

Pandocは文書の形式を変換するtoolです。Markdownから `.docx` やHTMLを作れます。たとえばWordへの変換は1行です。

```
pandoc report.md -o report.docx
```

「report.mdをWordにして」とCodexに頼むと、変換を実行できます。出てきたfileを開き、文字・数式・図を確かめます。

出力された `.docx` をWordで開くと、`#` が見出し1、`##` が見出し2のstyleとして入っています。ナビゲーションウィンドウに目次が出ます。

体裁を指定する

Wordの書式は見本の `.docx` で決まります。見出しのfontや余白を直した見本を用意し、「この見本の書式で変換して」とCodexに渡します。HTMLの体裁はCSS fileで決まります。

変換後のfileは直さない

`.docx` やHTMLを直接編集しても、次に変換したときに上書きされます。直す先は常に元のMarkdownです。変換は何度でもやり直せます。

コラム：Office appに入るClaudeとChatGPT

> 2026年9月時点で、ClaudeとChatGPTはMicrosoft Officeのadd-inを出しています。Microsoft Marketplaceから入れ、appの右側に出るpanelで各自のaccountにsign inして使います。
>
> | | Word | Excel | PowerPoint | Outlook | 使えるplan |
> |---|---|---|---|---|---|
> | Claude | ○ | ○ | ○ | ○ | Pro、Max、Team、Enterprise |
> | ChatGPT | × | ○ | ○ | × | Freeを含む全plan（Free・Goは制限あり） |
>
> Claude for Wordは、編集をWordの変更履歴として入れます。一つずつ承諾・却下でき、コメントに沿った修正もできます。ChatGPTにWord版はなく、WordでOpenAIのmodelを使うならMicrosoft 365 Copilotを経由します。
>
> この授業の流れでは、add-inで `.docx` を直すと、上の節と同じ問題が起きます。Markdownから作り直した時点で修正が消えます。add-inが向くのは、共著者から戻ってきた `.docx` の変更履歴を読む場面や、相手がWordで作業を続ける場面です。

PDFにする

PDFは、いったんHTMLにしてから、browserで印刷して作ります。体裁をCSSで書けるのが利点です。

手でやるなら、HTMLをChromeで開き、⌘P（WindowsはCtrl+P）から「PDFに保存」を選びます。何度も作り直すときは「report.mdをHTML経由でPDFにして」とCodexに頼みます。Chromeを画面なしで動かすcommandを組んで実行します。

CSSの `@media print` に書いた指定がここで効きます。図の大きさや改頁の位置を、画面用と印刷用で変えられます。

体裁にこだわらないなら、`.docx` に変換してWordから書き出す方法もあります。

変換が失敗したときのerror messageは、そのままAIに渡すと直せることが多いです。何を直したかは差分で見ます。

Marpでslideにする

仕組み

Marpは、Markdownからpresentation slideを作るtoolです。前半で書いた原稿と同じ記法のまま、頁に区切ったものがslideになります。

fileの先頭に `marp: true` を書きます。

```
---
marp: true
theme: default
paginate: true
size: 16:9
---

# 1枚目の題

- 箇条書き
- もう一つ

---

# 2枚目の題
```

`---` を単独の行に置くと、そこが頁の区切りです。front matterの `---` と同じ記号ですが、位置で区別されます。

書き出す

VS Codeの拡張（marp-team.marp-vscode）を入れると、編集しながら右側にslideのpreviewが出ます。

PDFやHTMLへの書き出しは、「slides.mdをPDFにして」とCodexに頼みます。Marpのcommandを組んで実行します。出てきたfileを開き、図が空欄になっていないかを見ます。

reportとslideは別のfile

同じfolderに `report.md` と `slides.md` を並べて、図は共通の `figures/` を見る構成にします。reportの文章をそのままslideに貼ると、1枚に収まりません。slideは箇条書きと図が中心になります。

補論　LaTeXについて

経済学の論文誌に出す原稿は、ほとんどがLaTeXで書かれています。この授業では使いませんが、卒論や大学院で必要になるかもしれないので、触れておきます。

Markdownとの違い

LaTeXは1980年代からある組版system（TeX）の上に作られた文書作成の仕組みです。Markdownと同じく、text fileに記号を書いて構造を示します。違うのは、Markdownが「変換先で体裁が決まる」のに対し、LaTeXは「組版の指定まで自分で書く」点です。そのぶん記法の量が多く、fileの先頭に設定（preamble）を並べます。

```
\documentclass[12pt,a4paper]{article}
\usepackage{amsmath,amssymb,amsfonts}
\usepackage{graphicx}
\usepackage{natbib}
\usepackage[margin=1in]{geometry}
```

数式の書き方は「数式」の節で見たものと同じです。Markdownで `$` に挟んで書いていたのは、もともとLaTeXの記法です。

主なもの

名前	何か
TeX / LaTeX	組版のsystem本体と、その上の文書作成の枠組み
TeX Live、MacTeX	一式をPCに入れるためのdistribution。数GBあります
pdflatex、lualatex	`.tex` からPDFを作るcommand。日本語にはlualatexを使います
BibTeX	参考文献のdatabase（`.bib`）から文献リストを作る
Overleaf	browserで書いてcompileできるservice。installが要りません

compileが要る

Markdownはpreviewがその場で出ますが、LaTeXは毎回compileします。しかも1回では足りません。

```
pdflatex main.tex
bibtex main
pdflatex main.tex
pdflatex main.tex
```

1回目が本文を組み、`\cite` で引いた文献のkeyを記録します。`bibtex` がそれと `.bib` を突き合わせて文献リストを作ります。3回目でそれが本文に入り、4回目で番号と相互参照が確定します。`latexmk -pdf main.tex` を使うと、必要な回数だけ自動で回ります。

この手間と引き換えに、番号付けを自分で書かずに済みます。表に `\label{tab:events}` を付けておくと、本文から `\ref{tab:events}` で呼べます。節を入れ替えても番号は付け直されます。図、表、式、定理、文献も同じ仕組みです。

参考文献

文献は `.bib` fileに貯めます。1件が1つのentryです。

```
@article{jorda2005,
  author  = {Jord{\`a}, {\`O}scar},
  title   = {Estimation and Inference of Impulse Responses by Local Projections},
  journal = {American Economic Review},
  volume  = {95}, number = {1}, pages = {161--182}, year = {2005}
}
```

本文には `\citep{jorda2005}` と書くだけです。出力の形（著者年か番号か、雑誌名を略すか）は style file（`.bst`）が決めます。投稿先を変えるときは style file を差し替えます。文献リストの並べ直しは要りません。

Google ScholarやJSTORには、文献のBibTeX形式を出す機能があります。貼り付けて貯めていく使い方になります。

Overleaf

Overleaf は、browserで動くLaTeXの編集画面です。TeXをPCに入れなくてよく、左に原稿、右にPDFが出ます。共著者と同時に編集でき、変更履歴も残ります。経済学では共著の原稿をここに置くのが普通になっています。

無料planでもcompileは通ります。compile時間の上限と、同時編集の人数に制限があります。

サンプル

`sample_latex/` に、第1回で見た日米自動車産業と為替の論文の原稿一式を置いてあります。AIに作らせた試作の論文です。`main.tex` が英語版の本文、`main_ja.tex` が日本語版の本文、`references.bib` が文献、`main.pdf` と `main_ja.pdf` が出来上がり（20ページと19ページ）です。図の4枚はPNGで、本文から `\includegraphics` で読み込んでいます。

TeXが入っていれば、そのfolderで上の4つのcommandを走らせると `main.pdf` ができます。日本語版は `pdflatex` を `lualatex` に替え、`main` を `main_ja` にします。入っていない場合は、folderをZIPにしてOverleafの「Upload Project」に投げると開けます。日本語版はOverleafのメニューでcompilerをLuaLaTeXに替えます。

この回の要点

- Markdownはtext、`.docx` はZIPで固めたXMLです。差分が取れるかどうかがここで決まります。
- PandocはMarkdownを `.docx`、HTML、PDFに変換します。体裁は変換のときに指定するので、原稿には構造だけ書きます。
- PDFはHTMLを経由して作ります。画面用と印刷用の体裁は、どちらもCSSで書きます。
- 論文誌の原稿はLaTeXです。この授業では使いませんが、番号付けと文献リストを自動で作る仕組みだけ見ておいてください。